

1. Objetivo

Hacer que los hilos de ejecución de un *kernel* trabajen de forma cooperativa.

2. Ejercicio

Calcular de forma aproximada el valor del número π a partir de la siguiente expresión:

$$\frac{\pi^2}{6} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \cdots + \frac{1}{n^2} + \cdots$$

Además de mostrar las propiedades del dispositivo (nombre, capacidad de cómputo, número de multiprocesadores, número de núcleos y máximo número de hilos por bloque) y el número de hilos lanzados, el programa debe cumplir los siguientes requisitos:

- Pedir por teclado un número de términos que sea potencia de 2.
- Generar cada término de la sucesión dentro de la función *kernel*.
- Sumar todos los términos utilizando un algoritmo de reducción paralela.
- Mostrar el valor obtenido y el error relativo tomando como referencia del valor de π el número 3,141593f.

3. Solución propuesta

```
/*
* ARQUITECTURA DE COMPUTADORES
* 2º Grado en Ingenieria Informatica
* Introduccion a la programacion en CUDA
*
* Basico 5: "Sincronizacion (Reduccion paralela)"
* >> Calcular el valor de pi a partir de una serie infinita:
*    pi^2 / 6 = 1/1^2 + 1/2^2 + 1/3^2 + 1/4^2 + 1/5^2 ...
*
*/
////////////////////////////////////////////////////////////////////////
// includes
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>
```

```
////////////////////////////////////////////////////////////////////////
// defines
#define PI 3.141593f // definimos PI como un dato de tipo "float"

////////////////////////////////////////////////////////////////////////
// declaracion de funciones
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ int propiedades_Device();
// Funcion que imprime las propiedades del device y devuelve el maximo nº de hilos por bloque

// GLOBAL: funcion llamada desde el host y ejecutada en el device (kernel)
__global__ void calculoPi(float *vector, float *pi)
// Funcion que calcula el numero pi
{
	// KERNEL CON 1 SOLO BLOQUE
	// identificacion de cada hilo
	int myID = threadIdx.x;
	// cada hilo genera su denominador
	float n = myID + 1;
	// cada hilo escribe en el vector un termino de la sucesion
	vector[myID] = 1.0 / (n*n);
	// sincronizamos todos los hilos para evitar riesgos RAW
	__syncthreads();

	// ******************
	// REDUCCION PARALELA
	// ******************
	int salto = blockDim.x / 2;
	while (salto > 0)
	{
		// en cada paso solo trabajan la mitad de los hilos
		if (myID < salto)
		{
			vector[myID] = vector[myID] + vector[myID + salto];
		}
		// sincronizamos los hilos evitar riesgos de tipo RAW
		__syncthreads();
		salto = salto / 2;
	}
	// ******************

	// Solo el hilo no.'0' escribe el resultado final:
	// evitamos los riesgos estructurales por el acceso a la memoria
	if (myID == 0)
	{
		*pi = sqrt(6 * vector[0]);
	}
}

////////////////////////////////////////////////////////////////////////
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{
	// buscando dispositivos
	int maxThreads;
	int deviceCount;
	cudaGetDeviceCount(&deviceCount);
	if (deviceCount == 0)
	{
		printf("!!!!!No se han encontrado dispositivos CUDA!!!!!\n");
		printf("pulsa [INTRO] para finalizar");
		getchar();
		return 1;
	}
	else
	{
		maxThreads = propiedades_Device();
	}

	// declaraciones
	float *hst_pi;
	float *dev_pi, *dev_vector;
	int terminos;


	// pedimos por teclado el numero de terminos de la sucesion
	printf("  Introduce el numero de terminos (potencia de 2): ");
	scanf("%d", &terminos);
	// capturamos el [INTRO] de la funcion scanf()
	getchar();
```

```
	while (terminos > maxThreads)
	{
		printf("> Se ha superado el maximo numero de hilos por bloque (%d hilos)\n",
maxThreads);
		printf("  Introduce de nuevo el numero de terminos (potencia de 2): ");
		scanf("%d", &terminos);
		// capturamos el [INTRO] de la funcion scanf()
		getchar();
	}

	// reserva en el host
	hst_pi = (float*)malloc(sizeof(float));

	// reserva en el device
	cudaMalloc((void**)&dev_pi, sizeof(float));
	cudaMalloc((void**)&dev_vector, terminos * sizeof(float));

	// DIMENSIONES DEL KERNEL
	// lanzamos 1 bloque
	printf("> Lanzamiento con 1 bloque de %d hilos\n", terminos);

	// llamada al kernel
	calculoPi <<< 1, terminos >>> (dev_vector, dev_pi);

	// recogida de datos desde el device
	cudaMemcpy(hst_pi, dev_pi, sizeof(float), cudaMemcpyDeviceToHost);

	// error relativo del calculo
	float error = 100 * (*hst_pi - PI) / PI;

	// impresion de resultados
	printf("> Valor de pi        : % f\n", PI);
	printf("> Valor calculado    : % f\n", *hst_pi);
	printf("> Error relativo     : %f%%\n", error);

	// salida del programa
	time_t fecha;
	time(&fecha);
	printf("************************************************\n");
	printf("Programa ejecutado el: %s\n", ctime(&fecha));
	printf("<pulsa INTRO para finalizar>");
	getchar();
	return 0;
}
////////////////////////////////////////////////////////////////////////
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ int propiedades_Device()
{
	int currentDevice;
	cudaDeviceProp deviceProp;
	cudaGetDevice(&currentDevice);
	cudaGetDeviceProperties(&deviceProp, currentDevice);
	// calculo del numero de cores (SP)
	int cudaCores = 0;
	int SM = deviceProp.multiProcessorCount;
	int major = deviceProp.major;
	int minor = deviceProp.minor;
	const char *archName;
	switch (major)
	{
	case 1:
		//TESLA
		archName = "TESLA";
		cudaCores = 8;
		break;
	case 2:
		//FERMI
		archName = "FERMI";
		if (minor == 0)
			cudaCores = 32;
		else
			cudaCores = 48;
		break;
	case 3:
		//KEPLER
		archName = "KEPLER";
		cudaCores = 192;
		break;
	case 5:
		//MAXWELL
		archName = "MAXWELL";
```

```
                cudaCores = 128;
                break;
        case 6:
                //PASCAL
                archName = "PASCAL";
                cudaCores = 64;
                break;
        case 7:
                //VOLTA (7.0) TURING (7.5)
                cudaCores = 64;
                if (minor == 0)
                        archName = "VOLTA";
                else
                        archName = "TURING";
                break;
        case 8:
                //AMPERE
                archName = "AMPERE";
                cudaCores = 64;
                break;
        default:
                //ARQUITECTURA DESCONOCIDA
                archName = "DESCONOCIDA";
                cudaCores = 0;
        }
        // presentacion de propiedades
        printf("************************************************\n");
        printf("DEVICE %d: %s\n", currentDevice, deviceProp.name);
        printf("************************************************\n");
        printf("> Capacidad de Computo            \t: %d.%d\n", major, minor);
        printf("> Arquitectura CUDA               \t: %s\n", archName);
        printf("> No. de MultiProcesadores        \t: %d\n", SM);
        printf("> No. de CUDA Cores (%dx%d)       \t: %d\n", cudaCores, SM, cudaCores*SM);
        printf("> No. maximo de Hilos (por bloque)\t: %d\n", deviceProp.maxThreadsPerBlock);
        printf("************************************************\n");

        // salida
        return deviceProp.maxThreadsPerBlock;
}
////////////////////////////////////////////////////////////////////////
```